Pointers

Luigi Ieracitano and LLMs

February 28, 2026

Contents

1 Pointers - 1 **5**

1.1 C04a — Introduction aux pointeurs : à quoi servent-ils? . . . 5

1.1.1 Démystifier les pointeurs . . . 5

1.1.2 Les trois usages fondamentaux des pointeurs . . . 5

1.2 C04b — Représentation concrète des pointeurs . . . 6

1.2.1 Un pointeur est une variable . . . 6

1.2.2 Analogie : carnet d'adresses . . . 7

1.2.3 Deux mondes : adresses vs valeurs . . . 7

1.2.4 Allouer un pointeur (allocation dynamique) . . . 7

1.2.5 Libérer un pointeur . . . 8

1.2.6 Affectation de pointeurs . . . 8

1.2.7 Le pointeur nul : `NULL` . . . 8

1.3 C04c — Utiliser les pointeurs en pratique en C . . . 9

1.3.1 Déclaration d'un pointeur . . . 9

1.3.2 L'opérateur & : obtenir une adresse . . . 9

1.3.3 L'opérateur * : accéder à la valeur pointée . . . 9

1.3.4 Relation fondamentale . . . 10

1.3.5 Modifier une valeur via un pointeur . . . 10

1.3.6 Pointeurs vers structures . . . 10

1.3.7 Attention au symbole * . . . 11

1.4 C04d — Pointeurs et fonctions : passage par "référence" . . . 11

1.4.1 Il n'existe pas de passage par référence en C . . . 11

1.4.2 Exemple : `scanf` . . . 11

1.4.3 Exemple classique : `swap` . . . 12

1.4.4 Optimisation : éviter les copies . . . 12

1.4.5 Retourner un pointeur : attention danger . . . 12

1.4.6 Solution : allocation dynamique . . . 13

1.4.7 Erreur fréquente à éviter . . . 13

1.5 C04e — Pointeurs et `const` . . . 13

1.5.1 Règle générale pour `const` . . . 13

1.5.2 Pointeur vers donnée constante . . . 13

1.5.3 Pointeur constant vers donnée modifiable . . . 14

1.5.4 Const sur les deux . . . 14

1.5.5 Exemple détaillé . . . 14

1.5.6 Point subtil . . . 15

1.5.7 Interprétation mentale utile . . . 15

1.6 C04f — Pointeurs vs références . . . 15

1.6.1 En C : pas de références . . . 15

1.6.2 Qu'est-ce qu'une référence ? . . . 15
1.6.3 Différences fondamentales . . . 16
1.6.4 Conséquences pratiques . . . 16
1.6.5 Lien avec les usages des pointeurs . . . 17
1.6.6 Idée finale . . . 17

2 Pointers - 2 **18**
2.1 C01b — Pointers : Introduction à l'allocation dynamique . . . 18
2.1.1 Rappel : les trois usages des pointeurs . . . 18
2.1.2 Gestion de la mémoire d'un processus . . . 18
2.1.3 Allocation statique (automatique) . . . 18
2.1.4 Limite de la pile . . . 19
2.1.5 Allocation dynamique (sur le tas) . . . 19
2.1.6 Idée fondamentale : durée de vie . . . 19
2.2 C01b — Pointers : malloc et calloc . . . 19
2.2.1 Allocation dynamique : `malloc`, `calloc`, `realloc` et `free` . . . 19
2.3 C01b — Pointers : realloc . . . 22
2.3.1 Libération de la mémoire : `free` . . . 22
2.4 C01b — Pointers : Allocation dynamique : exemple complet avec `malloc` et `free` . . . 24
2.4.1 1. Fonction de création . . . 24
2.4.2 2. Utilisation dans une autre fonction . . . 25
2.4.3 3. Égalité de pointeurs . . . 25
2.4.4 4. Durée de vie : ce qu'il faut absolument comprendre . . . 25
2.4.5 5. Attention aux erreurs classiques . . . 26
2.4.6 6. Schéma conceptuel temporel . . . 26
2.5 C01b — Pointers : Initialisation des pointeurs et bonnes pratiques . . . 26
2.5.1 1. Le cas dangereux : pointeur non initialisé . . . 26
2.5.2 2. Segmentation fault vs Bus error . . . 27
2.5.3 3. Bonne pratique : toujours initialiser . . . 27
2.5.4 4. Règles fondamentales de gestion mémoire . . . 27
2.5.5 5. Responsabilité : qui alloue doit libérer . . . 28
2.5.6 6. Toujours tester le retour de malloc / calloc . . . 28
2.5.7 7. malloc vs calloc . . . 28
2.5.8 8. Initialiser la mémoire . . . 28
2.5.9 9. Après free : effacer l'adresse . . . 29
2.5.10 10. Passage par pointeur et const . . . 29
2.6 C01b — Pointers : Étude de cas : implémentation d'un tableau dynamique (vector) . 29
2.7 C01b — Pointers : La fonction `realloc` . . . 32
2.8 C01b — Pointers : Étude de cas : implémentation complète du vector (constructeur, push, enlarge) . . . 34
2.8.1 1. Constantes et types . . . 35
2.8.2 2. Constructeur . . . 35
2.8.3 3. Destructeur . . . 35
2.8.4 4. Fonction interne : vector_enlarge . . . 36

2.8.5 5. Fonction vector_push . . . 37
2.8.6 6. Pourquoi un while et pas un if ? . . . 37
2.8.7 7. Logique complète du push . . . 38

3 Pointeurs - 3 **39**
3.1 Chaînes de caractères en C : principes fondamentaux . . . 39
3.1.1 1. Différence fondamentale avec Java . . . 39
3.1.2 2. Convention de terminaison . . . 39
3.1.3 3. Littéraux de chaînes . . . 39
3.1.4 4. Initialisation d'un tableau . . . 40
3.1.5 5. Pointeur n'est PAS string . . . 40
3.1.6 6. Allocation dynamique correcte . . . 40
3.1.7 7. Erreur classique . . . 40
3.1.8 8. Cas correct 1 : chaîne constante . . . 41
3.1.9 9. Cas correct 2 : chaîne modifiable . . . 41
3.1.10 10. Différence clé : copie vs pointeur . . . 41
3.1.11 11. Règle essentielle . . . 41
3.2 Fonctions utilitaires pour les chaînes de caractères . . . 42
3.2.1 Toujours utiliser les versions avec `n` . . . 42
3.2.2 Copie : `strncpy` . . . 42
3.2.3 Concaténation : `strncat` . . . 43
3.2.4 Comparaison : `strncmp` . . . 43
3.2.5 Longueur : `strlen` . . . 43
3.2.6 Recherche de caractères . . . 44
3.2.7 Recherche de sous-chaîne : `strstr` . . . 44
3.2.8 Entrées / sorties . . . 44
3.2.9 Lecture . . . 44
3.3 Résumé des bonnes pratiques . . . 45
3.4 Pointeurs sur fonctions et généricité . . . 45
3.4.1 Déclaration d'un pointeur sur fonction . . . 45
3.4.2 Affectation . . . 45
3.4.3 Appel via pointeur . . . 46
3.5 Passage de fonctions en argument . . . 46
3.6 Généricité et pointeurs génériques . . . 46
3.7 Exemple : tri générique avec qsort . . . 46
3.7.1 Pourquoi void* ? . . . 47
3.7.2 Exemple : tri d'un tableau d'entiers . . . 47
3.7.3 Fonction de comparaison générique . . . 47
3.7.4 Pourquoi faire cette conversion ? . . . 47
3.8 Comparaison pour des doubles . . . 47
3.9 Principe fondamental de la généricité en C . . . 48
3.10 Points importants . . . 48
3.11 Lecture des types complexes en C . . . 48
3.11.1 Règle fondamentale de lecture . . . 48

3.12 Exemple de type complexe . . . 49
3.12.1 Étape 1 : partir du nom . . . 49
3.12.2 Étape 2 : que retourne f ? . . . 49
3.12.3 Étape 3 : vers quoi pointe ce pointeur ? . . . 49
3.12.4 Étape 4 : contenu du tableau . . . 49
3.12.5 Étape 5 : que retournent ces fonctions ? . . . 50
3.13 Lecture complète . . . 50
3.14 Conseil pratique essentiel . . . 50
3.15 Réécriture avec typedef . . . 50
3.16 Casting en C : valeurs et pointeurs . . . 51
3.17 Casting de valeurs . . . 51
3.18 Casting de pointeurs . . . 51
3.18.1 Ce qui se passe réellement . . . 52
3.19 Différence fondamentale . . . 52
3.20 Casting et code générique . . . 52
3.21 Casting de pointeurs sur fonctions . . . 52
3.22 Casting et void* . . . 53
3.22.1 Erreur courante avec malloc . . . 53

4 Pointers - 4 **54**
4.1 Pointeurs et tableaux . . . 54
4.2 Accès via la syntaxe tableau . . . 54
4.3 Différence fondamentale : pointeur vs tableau . . . 54
4.4 Mémoire : stack vs heap . . . 55
4.5 Cas 1 : tableau bidimensionnel . . . 55
4.6 Cas 2 : tableau de pointeurs . . . 55
4.7 Cas 3 : double pointeur . . . 56
4.8 Différences cruciales . . . 56
4.8.1 Continuité mémoire . . . 56
4.8.2 Allocation . . . 56
4.8.3 Dimensions variables . . . 56
4.9 Attention importante . . . 56
4.10 Arithmétique des adresses . . . 57
4.11 Résumé final . . . 57

Pointers - 1

1.1 C04a — Introduction aux pointeurs : à quoi servent-ils?

1.1.1 Démystifier les pointeurs

Les pointeurs sont souvent perçus comme un concept difficile en C. En réalité, leur idée fondamentale est simple.

Un pointeur est simplement :

- une **référence vers une donnée en mémoire**
- un moyen d'accéder à un objet sans le copier

On peut comparer un pointeur à un **lien** ou un **signet** :

- quand on partage un site web, on ne copie pas tout le site
- on partage seulement un lien vers ce site

De la même manière, un pointeur est un lien vers une donnée. Il permet d'accéder à cette donnée depuis différents endroits du programme sans la dupliquer.

1.1.2 Les trois usages fondamentaux des pointeurs

Les pointeurs servent principalement à trois choses.

1. Partage d'un objet (référence)

On peut vouloir utiliser le même objet à plusieurs endroits du programme.

- éviter la duplication de données
- partager une structure entre fonctions
- permettre à plusieurs fonctions de modifier le même objet

Dans ce cas, le pointeur agit comme une **référence vers l'objet**. On manipule toujours le même objet en mémoire.

2. Généricité

Parfois, on ne sait pas à l'avance quel objet on veut manipuler.

Un pointeur permet alors :

- de décrire un objet **de façon générique**
- de choisir dynamiquement l'objet utilisé
- d'écrire du code réutilisable

Exemple : pointeurs vers fonctions Supposons que l'on possède plusieurs fonctions mathématiques:

$$f_1(x),\ f_2(x),\ f_3(x),\ \ldots$$

On veut écrire une fonction qui calcule une intégrale, mais la fonction à intégrer est choisie par l'utilisateur.

Sans pointeurs :

- il faudrait recompiler le programme pour changer la fonction

Avec un pointeur vers fonction :

- on choisit dynamiquement la fonction
- on passe un pointeur vers la fonction choisie

Le pointeur devient alors une description générique de la fonction à manipuler.

3. Allocation dynamique (contrôle de la durée de vie)

Les pointeurs permettent aussi de gérer la durée de vie des objets.

On distingue deux notions importantes :

- **Portée (scope)** : ensemble des lignes où une variable est visible dans le code
- **Durée de vie** : intervalle de temps où l'objet existe réellement en mémoire

Pour une variable locale classique :

- elle est créée au début du bloc
- elle est détruite à la fin du bloc

Sa durée de vie correspond donc à la durée d'exécution de sa portée.

Cependant, on peut vouloir :

- créer un objet dans une fonction
- continuer à l'utiliser après le retour de la fonction
- contrôler manuellement sa création et sa destruction

Dans ce cas, on utilise l'**allocation dynamique** (qui sera étudiée en détail plus tard).

Les pointeurs permettent alors d'accéder à cet objet au-delà de sa portée syntaxique.

1.2 C04b — Représentation concrète des pointeurs

1.2.1 Un pointeur est une variable

En C, un pointeur est une variable comme les autres.

- il occupe un emplacement en mémoire
- il possède une valeur stockée

La seule différence est que cette valeur n'est pas un nombre ordinaire, mais une **adresse mémoire**.

Autrement dit :

un pointeur stocke l'adresse d'une autre variable

On peut voir un pointeur comme une **variable qui contient l'adresse d'une variable**.

Cette adresse permet d'accéder à la zone mémoire correspondante et donc à la valeur de l'objet pointé.

1.2.2 Analogie : carnet d'adresses

Une bonne façon de comprendre les pointeurs est de les voir comme des pages dans un carnet d'adresses.

- une **maison** représente une donnée en mémoire
- une **adresse** correspond à l'emplacement mémoire
- une **page du carnet** correspond à un pointeur

Le pointeur ne contient pas la maison, il contient seulement son adresse.

Conséquence importante Déclarer un pointeur ne crée pas l'objet pointé.

- on ajoute simplement une nouvelle page dans le carnet
- cette page peut être vide
- elle peut contenir une adresse invalide

Un pointeur non initialisé ne pointe vers rien de fiable.

1.2.3 Deux mondes : adresses vs valeurs

Avec les pointeurs, il est essentiel de distinguer deux mondes :

- le **monde des valeurs**
- le **monde des adresses**

La plupart des erreurs viennent de la confusion entre ces deux niveaux.

$$\textbf{valeur pointée} \neq \textbf{pointeur}$$

Le pointeur contient une adresse, la valeur pointée se trouve ailleurs en mémoire.

Chaque fois que vous manipulez un pointeur, vous devriez vous demander :

Suis-je en train de manipuler une adresse ou une valeur ?

1.2.4 Allouer un pointeur (allocation dynamique)

Dire que l'on "alloue un pointeur" est en réalité un abus de langage.

Ce que l'on fait réellement est :

1. créer un objet quelque part en mémoire
2. stocker son adresse dans le pointeur

Dans l'analogie :

- on construit une maison
- puis on écrit son adresse dans le carnet

L'allocation agit donc sur la **valeur pointée**, pas sur le pointeur lui-même.

1.2.5 Libérer un pointeur

Là encore, l'expression est trompeuse.

On ne libère pas le pointeur, on libère la mémoire pointée.

- la zone mémoire est rendue au système
- le pointeur conserve toujours l'adresse

Dans l'analogie :

- on détruit la maison
- mais l'adresse reste écrite dans le carnet

Le pointeur devient alors **dangling** (il contient une adresse invalide).

C'est pourquoi, après une libération, il est recommandé de mettre le pointeur à `NULL`.

1.2.6 Affectation de pointeurs

Considérons l'instruction :

```
p1 = p2;
```

Cela signifie simplement :

copier une adresse

On ne copie pas la valeur pointée, on copie uniquement l'adresse.

Dans l'analogie :

- quelqu'un vous donne une adresse
- vous la recopiez dans votre carnet

Les deux pointeurs désignent alors le même objet.

1.2.7 Le pointeur nul : `NULL`

L'instruction :

```
p = NULL;
```

signifie que le pointeur ne contient aucune adresse valide.

- cela n'affecte pas la mémoire pointée auparavant
- cela efface seulement l'adresse stockée

Dans l'analogie :

- on gomme la page du carnet
- la maison existe peut-être encore

Il est crucial de ne pas confondre :

- `free(p)` : libère la mémoire pointée
- `p = NULL` : efface l'adresse

1.3 C04c — Utiliser les pointeurs en pratique en C

1.3.1 Déclaration d'un pointeur

Un pointeur est une variable, mais son type indique la nature des données pointées.
La syntaxe générale est :

```
type *nom;
```

Exemple :

```
int *px;
```

Cela signifie :

- `px` est un pointeur
- il pointe vers une zone mémoire contenant des `int`

Le type du pointeur indique donc comment lire la mémoire pointée.

Bonne pratique Comme toute variable, un pointeur devrait être initialisé.

```
int *px = NULL;
```

Cela garantit qu'il ne contient pas une adresse aléatoire.

1.3.2 L'opérateur & : obtenir une adresse

L'opérateur `&` permet de passer du monde des valeurs au monde des adresses.

```
int x = 3;
int *px = &x;
```

Ici :

- `&x` retourne l'adresse de `x`
- cette adresse est stockée dans `px`
- `px` pointe maintenant vers `x`

On peut visualiser :

$$\texttt{px} \rightarrow \texttt{x} = 3$$

1.3.3 L'opérateur * : accéder à la valeur pointée

L'opérateur `*` permet de passer du monde des adresses au monde des valeurs.

```
int x = 3;
int *px = &x;
printf("%d\n", *px);
```

- `px` contient une adresse
- `*px` lit la valeur à cette adresse

L'instruction affiche donc :

3

Conseil mental Au début, verbalisez :

px = contenu pointé par px (déréférencement)

(lors de l'utilisation pas à la déclaration)
Cela aide à ne pas confondre adresse et valeur.

1.3.4 Relation fondamentale

Pour toute variable `x` :

```
*(&x) == x
```

Pourquoi ?

- `&x` donne l'adresse de `x`
- `*` lit le contenu à cette adresse

On retombe donc sur la valeur initiale.

1.3.5 Modifier une valeur via un pointeur

Un pointeur permet aussi de modifier directement la variable pointée.

```
int x = 3;
int *px = &x;

*px = 10;
```

Après cette instruction :

`x == 10`

On a modifié la variable originale via son adresse.

1.3.6 Pointeurs vers structures

Si un pointeur pointe vers une structure :

```
struct Point {
    int x;
    int y;
};

struct Point p = {1,2};
struct Point *ptr = &p;
```

On peut accéder aux champs de deux façons :

```
(*ptr).x
```

ou, plus simplement :

```
ptr->x
```

La notation `->` est donc un raccourci pour :

`(*ptr).champ`

1.3.7 Attention au symbole *

Le symbole `*` a plusieurs rôles en C :

- multiplication
- déclaration de pointeur
- accès à la valeur pointée

Il faut bien distinguer :

- `int *p;` déclaration
- `*p` accès mémoire

La déclaration n'est exécutée qu'une fois, mais l'accès mémoire peut apparaître partout dans le programme.

1.4 C04d — Pointeurs et fonctions : passage par "référence"

1.4.1 Il n'existe pas de passage par référence en C

En C, il n'existe que le passage par valeur.
Même lorsqu'on parle de "passage par référence", on fait en réalité :

un passage par valeur de pointeur

Pourquoi ?

- un pointeur est une adresse
- si on passe une adresse à une fonction
- la fonction peut modifier le contenu à cette adresse

On obtient donc un effet équivalent au passage par référence.

1.4.2 Exemple : `scanf`

Considérons :

```
int x;
scanf("%d", &x);
```

Pourquoi met-on `&x` ?

- `scanf` doit modifier `x`
- pour modifier une variable, il faut connaître son adresse
- on passe donc l'adresse de `x`

Sans `&`, la modification serait impossible.

1.4.3 Exemple classique : `swap`

Supposons :

```
int i = 3;
int j = 2;
swap(&i, &j);
```

On veut inverser les contenus mémoire de `i` et `j`.

La fonction correcte est :

```
void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Les trois ingrédients du "passage par référence" :

1. Les paramètres sont des pointeurs
2. On utilise `*` pour accéder aux valeurs pointées
3. On passe les arguments avec `&`

1.4.4 Optimisation : éviter les copies

Passer un objet volumineux par valeur peut être coûteux.

Exemple : matrices 100x100.

Solution :

- passer un pointeur
- éviter la copie

Mais si la fonction ne doit pas modifier l'objet, on protège avec `const` :

```
Complex add(const Complex *a, const Complex *b);
```

Cela signifie :

- on passe des adresses (pas de copie)
- la fonction n'a pas le droit de modifier le contenu

Conseil pratique Quand vous passez un pointeur à une fonction :

mettez systématiquement `const` sauf si modification voulue

1.4.5 Retourner un pointeur : attention danger

Considérons :

```
int* f() {
    int x = 5;
    return &x;
}
```

Ceci est **incorrect**.

Pourquoi ?

- `x` est une variable locale
- sa durée de vie s'arrête à la fin de la fonction
- on retourne l'adresse d'une zone mémoire invalide

C'est un pointeur vers une variable qui n'existe plus.

1.4.6 Solution : allocation dynamique

Si on veut qu'un objet survive après la fin de la fonction, il faut utiliser l'allocation dynamique :

```
int* f() {
    int *p = malloc(sizeof(int));
    *p = 5;
    return p;
}
```

Ici :

- la mémoire est allouée dynamiquement
- sa durée de vie dépasse celle de la fonction

Mais :

tout malloc doit être accompagné d'un free

1.4.7 Erreur fréquente à éviter

Ne jamais faire :

- retourner l'adresse d'une variable locale
- oublier de libérer une mémoire allouée dynamiquement

Les pointeurs exigent une gestion rigoureuse de la durée de vie des objets.

1.5 C04e — Pointeurs et `const`

1.5.1 Règle générale pour `const`

La règle pour comprendre `const` en C est simple :

`const` s'applique à ce qui précède, sauf s'il n'y a rien avant, alors il s'applique à ce qui suit.

Cette règle permet d'interpréter toutes les déclarations impliquant des pointeurs et `const`.

1.5.2 Pointeur vers donnée constante

Considérons :

```
const int *p;
```

ou équivalent :

```
int const *p;
```

Ici, `const` s'applique au type `int`.
Cela signifie :

- le pointeur peut changer d'adresse
- mais la valeur pointée ne peut pas être modifiée via ce pointeur

Autrement dit :

p est modifiable, mais *p est en lecture seule

1.5.3 Pointeur constant vers donnée modifiable

Considérons :

```
int * const p;
```

Ici, `const` s'applique au pointeur lui-même.
Cela signifie :

- le pointeur ne peut pas changer d'adresse
- mais la valeur pointée peut être modifiée

Autrement dit :

p est constant, mais *p est modifiable

1.5.4 Const sur les deux

On peut combiner les deux :

```
const int * const p;
```

Cela signifie :

- le pointeur est constant
- la valeur pointée est constante

On ne peut modifier ni l'adresse ni le contenu.

1.5.5 Exemple détaillé

Considérons :

```
int i = 2;
int j = 3;

const int *p1 = &i;
int * const p2 = &i;
```

Comportement de `p1`

- `*p1 = 5;` interdit
- `p1 = &j;` autorisé

`p1` peut pointer ailleurs, mais ne peut pas modifier le contenu pointé.

Comportement de `p2`

- `*p2 = 5;` autorisé
- `p2 = &j;` interdit

`p2` pointe toujours vers la même adresse, mais peut modifier la valeur pointée.

1.5.6 Point subtil

Le fait qu'un pointeur pointe vers un objet constant ne signifie pas que cet objet ne peut jamais changer.
Cela signifie seulement :

cet objet ne peut pas être modifié au travers de ce pointeur

Il peut être modifié par un autre pointeur non-const ou directement via son nom.

1.5.7 Interprétation mentale utile

On peut résumer ainsi :

- `const int *p` → pointeur vers lecture seule
- `int * const p` → pointeur figé
- `const int * const p` → tout figé

Comprendre cette distinction est essentiel pour écrire des interfaces sûres et éviter des modifications involontaires de données.

1.6 C04f — Pointeurs vs références

1.6.1 En C : pas de références

En C, il n'existe pas de références.
Le langage ne possède que :

- des valeurs
- des pointeurs

Lorsque l'on parle de "passage par référence" en C, il s'agit toujours d'un passage par valeur de pointeur.
Les références existent en revanche dans d'autres langages, notamment en C++.

1.6.2 Qu'est-ce qu'une référence ?

Une référence peut être vue comme un **autre nom** donné à une variable existante.
Contrairement à un pointeur :

- une référence n'est pas une zone mémoire
- elle ne stocke pas une adresse
- elle est simplement un alias géré par le compilateur

On peut l'imaginer comme :

un drapeau posé sur une variable existante

Le drapeau ne crée pas une nouvelle mémoire, il désigne simplement une mémoire déjà existante.

1.6.3 Différences fondamentales

1. Existence en mémoire

- Un pointeur existe réellement en mémoire
- Une référence peut ne pas exister comme objet mémoire distinct

Un pointeur est une variable, une référence est un alias.

2. Initialisation

- Un pointeur peut être non initialisé
- Une référence doit obligatoirement être initialisée

Une référence ne peut jamais être vide.

3. Valeur nulle

- Un pointeur peut valoir `NULL`
- Une référence ne peut jamais référencer "rien"

Elle est toujours attachée à un objet valide.

4. Réaffectation

- Un pointeur peut être redirigé vers un autre objet
- Une référence ne peut jamais changer d'objet

Une référence désigne toujours le même objet.

5. Niveaux d'indirection

- On peut avoir des pointeurs vers pointeurs
- Les références ne s'enchaînent pas de cette manière

Les pointeurs permettent donc des structures beaucoup plus flexibles.

1.6.4 Conséquences pratiques

Les références sont plus simples à manipuler :

- pas de gestion explicite d'adresses
- pas de pointeur nul
- moins d'erreurs possibles

C'est pourquoi, dans les langages qui offrent les deux :

on utilise des références quand c'est possible

et

des pointeurs quand c'est nécessaire

1.6.5 Lien avec les usages des pointeurs

On a vu que les pointeurs servent principalement à :

1. partager un objet
2. écrire du code générique
3. gérer l'allocation dynamique

Les références ne couvrent que le premier cas.

- elles ne permettent pas la généricité via redirection
- elles ne permettent pas la gestion explicite de la mémoire

Les pointeurs restent donc indispensables en C.

1.6.6 Idée finale

On peut voir une référence comme :

un pointeur automatique que l'on ne peut pas modifier

Tout est géré par le compilateur, ce qui simplifie l'usage mais réduit la flexibilité.

En C, puisqu'il n'existe pas de références, les pointeurs doivent être utilisés dans tous les cas où l'on souhaite partager, manipuler ou contrôler des données en mémoire.

Pointers - 2

2.1 C01b — Pointers : Introduction à l'allocation dynamique

2.1.1 Rappel : les trois usages des pointeurs

Lors du premier cours sur les pointeurs, nous avons vu trois usages principaux :

1. **Passage par référence** Permettre à une fonction de modifier une variable définie à l'extérieur.
2. **Description générique d'objets** Manipuler des objets dont la nature exacte n'est pas connue au moment de l'écriture du code.
3. **Allocation dynamique** Gérer nous-mêmes la durée de vie d'une zone mémoire.

Les deux premiers usages ont été détaillés précédemment. Nous abordons maintenant le troisième : **l'allocation dynamique**.

2.1.2 Gestion de la mémoire d'un processus

Pour comprendre l'allocation dynamique, il faut d'abord comprendre **l'organisation mémoire d'un programme**.

La mémoire virtuelle d'un processus peut être schématisée ainsi :

- En bas : le **code du programme** + constantes.
- Puis : les **variables globales et statiques**.
- Ensuite : le **tas (heap)**.
- En haut : la **pile (stack)**.

La pile et le tas évoluent en sens opposés :

- La **pile** grandit vers les adresses décroissantes.
- Le **tas** grandit vers les adresses croissantes.

2.1.3 Allocation statique (automatique)

Jusqu'à présent, toutes les variables déclarées dans vos programmes C étaient allouées automatiquement sur la **pile**.

Exemple :

```
void f() {
    int i = 5;
}
```

Lorsque la fonction commence :

- de la mémoire est réservée pour `i` sur la pile.

Lorsque la fonction se termine :

- la mémoire est automatiquement libérée.

On parle d'**allocation automatique (statique du point de vue du programmeur)**.

Le programmeur ne contrôle pas la durée de vie exacte : elle correspond strictement à la portée (*scope*) de la variable.

2.1.4 Limite de la pile

La pile a une taille limitée (souvent quelques mégaoctets).

Sous Unix/Linux, on peut vérifier la taille maximale avec :

```
ulimit -s
```

Si l'on alloue des structures trop volumineuses sur la pile, on risque un **stack overflow**.

2.1.5 Allocation dynamique (sur le tas)

Lorsque l'on veut :

1. Contrôler la **durée de vie** d'un objet au-delà de la portée,
2. Allouer de **grandes quantités de mémoire**,

il faut utiliser le **tas (heap)**.

Contrairement à la pile :

- L'allocation est faite explicitement par le programmeur.
- La désallocation doit aussi être faite explicitement.
- La mémoire disponible est beaucoup plus grande.

Le tas est limité uniquement par la mémoire virtuelle disponible (et bien sûr la mémoire physique réelle).

2.1.6 Idée fondamentale : durée de vie

La notion clé ici est la **durée de vie d'un objet en mémoire**.

- Sur la pile : durée de vie = durée de la portée.
- Sur le tas : durée de vie = décidée par le programmeur.

L'allocation dynamique permet donc de dissocier :

$$\text{durée de vie} \neq \text{durée de la portée}$$

C'est précisément le troisième grand usage des pointeurs.

2.2 C01b — Pointers : malloc et calloc

2.2.1 Allocation dynamique : `malloc`, `calloc`, `realloc` et `free`

Cycle de vie d'une allocation dynamique

Lorsque l'on décide de gérer dynamiquement une zone mémoire, son cycle de vie est le suivant :

$$\text{Allocation} \longrightarrow \text{Utilisation} \longrightarrow \text{Libération}$$

Les fonctions principales sont :

- `malloc`
- `calloc`
- `realloc`
- `free`

`malloc`

Prototype :

```
void* malloc(size_t size);
```

`malloc` réserve `size` octets sur le tas et retourne un pointeur vers le début de la zone allouée.

Exemple :

```
double* p = malloc(sizeof(double));
```

On utilise généralement l'opérateur :

```
sizeof(type)
```

qui retourne la taille en octets du type donné.

Important :

- La mémoire retournée par `malloc` n'est PAS initialisée.
- Son contenu est indéterminé.

`calloc`

Prototype :

```
void* calloc(size_t nmemb, size_t size);
```

`calloc` alloue :

$$nmemb \times size$$

octets consécutifs en mémoire.

Exemple :

```
double* p = calloc(3, sizeof(double));
```

Cela alloue la place pour 3 doubles consécutifs.

Différences avec `malloc` :

- `calloc` protège contre les erreurs d'overflow dans la multiplication.
- `calloc` initialise la mémoire à 0.

Pourquoi préférer `calloc` ? (Bug OpenSSH)

Considérons le code suivant (simplifié) :

```
int nresp = get_number();
char** response;

if (nresp > 0) {
    response = malloc(nresp * sizeof(char*));
    for (int i = 0; i < nresp; i++) {
        response[i] = get_packet();
    }
}
```

Supposons :

$$nresp = 1\,073\,741\,856$$

Sur une architecture 32 bits :

- `sizeof(char*) = 4`

- Donc on calcule :

$$1\,073\,741\,856 \times 4$$

Cette valeur dépasse la capacité représentable sur 32 bits. Elle déborde et devient 128.

Résultat :

- On alloue 128 octets seulement.
- La boucle itère jusqu'à 1 milliard.
- On écrit largement hors de la zone allouée.

C'est un **buffer overflow**, faille de sécurité réelle observée dans OpenSSH.

Solution :

```
response = calloc(nresp, sizeof(char*));
```

`calloc` détecte l'overflow interne et évite ce problème.

Initialisation mémoire : `memset`

Si l'on utilise `malloc` mais que l'on souhaite initialiser :

```
#include <string.h>

memset(ptr, value, size);
```

Exemple :

```
int* p = malloc(sizeof(int));
memset(p, 0, sizeof(int));
```

Cela met tous les octets à 0.

Important :

- `memset` agit au niveau des octets.
- Mettre tous les bits à 0 correspond à la valeur 0 pour les types entiers et les pointeurs nuls.

Vérification obligatoire

`malloc` et `calloc` retournent `NULL` si l'allocation échoue.

Toujours écrire :

```
double* p = calloc(3, sizeof(double));
if (p == NULL) {
    // gérer l'erreur
    exit(EXIT_FAILURE);
}
```

Ne jamais utiliser un pointeur sans avoir vérifié qu'il n'est pas nul.

`free`

Lorsque la zone mémoire n'est plus nécessaire :

```
free(p);
```

Après un `free`, le pointeur devient invalide. Bonne pratique :

```
free(p);
p = NULL;
```

Résumé

Fonction	Initialise ?	Protège overflow ?
malloc	Non	Non
calloc	Oui (0)	Oui

Principes essentiels :

- Toujours utiliser `sizeof`.
- Toujours vérifier le retour.
- Toujours libérer avec `free`.
- Préférer `calloc` si possible.

2.3 C01b — Pointers : realloc

2.3.1 Libération de la mémoire : `free`

Fin de la durée de vie

Dans le cycle de vie d'une allocation dynamique :

$$\text{Allocation} \longrightarrow \text{Utilisation} \longrightarrow \text{Libération}$$

après avoir utilisé une zone mémoire allouée avec `malloc`, `calloc` ou `realloc`, il faut la rendre au système.
C'est le rôle de :

```
free(ptr);
```

Ne pas libérer la mémoire entraîne une **fuite mémoire** (*memory leak*) :

- Le programme consomme de plus en plus de mémoire.
- La mémoire n'est jamais rendue au système.
- Sur le long terme : ralentissement, crash.

Ce que fait réellement `free`

Attention à un abus de langage courant :

On ne libère pas le pointeur.

On libère **la zone mémoire pointée**.
Schéma conceptuel :

- Monde des adresses : le pointeur contient une adresse.
- Monde des valeurs : la zone mémoire contient des données.

Lorsque l'on fait :

```
free(ptr);
```

- La zone mémoire est rendue au système.
- Mais le pointeur contient toujours la même adresse.

Le pointeur devient alors un **dangling pointer** (pointeur pendu).
Utiliser ce pointeur après un `free` entraîne un comportement indéfini.

Bonne pratique essentielle

Toujours écrire :

```
free(ptr);
ptr = NULL;
```

Pourquoi ?

- On rend la mémoire.
- On efface l'adresse.
- On évite toute utilisation accidentelle.

Un pointeur nul est sûr : toute tentative d'accès provoquera une erreur détectable.

Règle fondamentale

Toute allocation dynamique doit avoir un `free` correspondant.

Exemples :

- 1 appel à `malloc` → 1 appel à `free`
- 25 appels à `calloc` dans une boucle → 25 appels à `free`

Sinon : fuite mémoire.

Erreur classique

```
int* p = malloc(sizeof(int));
...
free(p);
*p = 5;   // ERREUR : accès après free
```

C'est un accès à une zone mémoire que l'on n'a plus le droit d'utiliser.

Résumé

- `free` libère la zone pointée, pas le pointeur.
- Toujours mettre le pointeur à `NULL` après.
- Toute allocation dynamique doit être libérée.
- Les fuites mémoire sont des erreurs graves en programmation système.

2.4 C01b — Pointers : Allocation dynamique : exemple complet avec `malloc` et `free`

Nous considérons ici le cas où l'on souhaite créer un entier dont la durée de vie dépasse la portée d'une fonction. Il s'agit du cas classique où une fonction alloue dynamiquement une zone mémoire et retourne un pointeur vers celle-ci.

L'objectif est de comprendre clairement la différence entre :

- la durée de vie des **pointeurs** (variables locales),
- la durée de vie de la **zone mémoire pointée**.

2.4.1 1. Fonction de création

La fonction suivante crée dynamiquement un entier et retourne son adresse.

```
#include <stdio.h>
#include <stdlib.h>

int* createLongLiveInt(void) {
    int* px = NULL;              // px existe uniquement dans cette fonction

    px = malloc(sizeof(int));    // allocation dynamique

    if (px != NULL) {
        *px = 20;                // initialisation de la zone mémoire
    }

    return px;                   // on retourne l'adresse allouee
}
```

Analyse détaillée

- `px` est un pointeur local : il n'existe que dans `createLongLiveInt`.
- `malloc(sizeof(int))` alloue une zone mémoire capable de contenir un entier.
- `malloc` retourne l'adresse de cette zone.
- Cette adresse est copiée dans `px`.
- On écrit ensuite dans la zone pointée via `*px = 20`.
- Lors du `return px`, on retourne une copie de l'adresse.

Important : à la fin de la fonction, la variable `px` disparaît, mais **la zone mémoire allouée continue d'exister** tant qu'elle n'est pas libérée avec `free`.

2.4.2 2. Utilisation dans une autre fonction

Voyons maintenant comment cette zone peut être utilisée ailleurs :

```c
void exampleUsage(void) {
    int* q = createLongLiveInt();   // q recoit l'adresse

    if (q != NULL) {
        *q = 123;                   // modification de la meme zone
    }

    int* r = q;                     // copie d'adresse (egalite de pointeurs)

    printf("%d\n", *r);             // acces via r

    free(q);                        // liberation de la zone
    q = NULL;                       // bonne pratique
    r = NULL;                       // eviter pointeur pendu
}
```

2.4.3 3. Égalité de pointeurs

L'instruction :

$$r = q;$$

ne copie pas la valeur stockée dans la mémoire pointée, mais uniquement l'adresse.

Ainsi :

- `q` et `r` pointent vers la même zone mémoire.
- Modifier `*q` modifie ce que voit `r`.

Il s'agit d'une **copie d'adresse**, pas d'une copie de contenu.

2.4.4 4. Durée de vie : ce qu'il faut absolument comprendre

La gestion de la durée de vie concerne :

la zone mémoire allouée, **pas les pointeurs**.

- `px` disparaît à la fin de `createLongLiveInt`.
- `q` disparaît à la fin de `exampleUsage`.
- Mais la zone mémoire reste valide tant qu'on n'a pas fait `free`.

La séquence correcte est donc :

`malloc` → utilisation → `free`

C'est le programmeur qui décide explicitement de la durée de vie.

2.4.5 5. Attention aux erreurs classiques

1. Oublier le `free` Provoque une fuite mémoire (memory leak).

2. Pointeur = NULL après `free` Après :

```
free(q);
```

la zone mémoire est libérée, mais l'adresse contenue dans `q` reste inchangée. Le pointeur devient un **dangling pointer**.
Toujours faire :

```
q = NULL;
```

2.4.6 6. Schéma conceptuel temporel

On peut résumer ainsi :

- Allocation dans `createLongLiveInt`
- Transmission de l'adresse par copie
- Utilisation via plusieurs pointeurs
- Libération explicite avec `free`

La zone mémoire existe indépendamment des variables locales qui ont servi à transmettre son adresse.

Conclusion :
L'allocation dynamique permet de créer des objets dont la durée de vie n'est pas limitée à la portée d'une fonction. Elle donne un contrôle total au programmeur, mais impose une discipline stricte :

Chaque malloc doit correspondre à un free

2.5 C01b — Pointers : Initialisation des pointeurs et bonnes pratiques

Un des pièges classiques avec les pointeurs est l'utilisation de zones mémoire non valides, en particulier des pointeurs non initialisés.

2.5.1 1. Le cas dangereux : pointeur non initialisé

Considérons le code suivant :

```c
#include <stdio.h>

void badExample(void) {
    int* px;          // NON initialise

    // ... beaucoup de code ici ...

    *px = 20;         // Ecriture dans une zone inconnue
}
```

Que se passe-t-il ici ?

- `px` est déclaré mais jamais initialisé.
- Il contient donc une valeur indéterminée.
- L'instruction `*px = 20;` tente d'écrire à une adresse inconnue.

Le programme compile parfaitement (aucune erreur syntaxique), mais à l'exécution il y a de fortes chances d'obtenir :

Segmentation fault

Pourquoi ?

Parce que vous accédez à une zone mémoire à laquelle vous n'avez pas le droit d'accéder.

2.5.2 2. Segmentation fault vs Bus error

Segmentation fault Se produit lorsque :

- vous accédez à une zone mémoire interdite,
- le noyau détecte directement l'accès invalide.

Bus error Plus rare. Peut apparaître lorsque :

- l'adresse pointe vers une zone mal alignée,
- ou vers de la mémoire matérielle spécifique (périphérique),
- et que l'erreur est détectée au niveau du bus matériel.

Dans les deux cas : c'est un accès mémoire invalide.

2.5.3 3. Bonne pratique : toujours initialiser

Il est fortement recommandé d'écrire :

```
int* px = NULL;
```

Pourquoi ?

Cela ne supprime pas le bug si vous oubliez d'allouer, mais garantit que le programme plantera exactement à l'endroit où vous utilisez le pointeur.

Exemple :

```
int* px = NULL;
*px = 20;   // Segmentation fault immediate
```

Cela rend le débogage beaucoup plus simple.

2.5.4 4. Règles fondamentales de gestion mémoire

Règle 1 : Toute allocation doit être libérée

$$\boxed{\texttt{malloc} \longleftrightarrow \texttt{free}}$$

Exemple correct :

```
int* px = malloc(sizeof(int));
if (px != NULL) {
    *px = 10;
}
free(px);
px = NULL;
```

Sinon : fuite mémoire (*memory leak*).

2.5.5 5. Responsabilité : qui alloue doit libérer

Principe fondamental :

- Si une fonction alloue, elle doit documenter clairement qui est responsable du `free`.
- Idéalement : fournir une fonction de destruction.

Exemple conceptuel :

```
Matrix* createMatrix(int n);
void destroyMatrix(Matrix* m);
```

Toujours proposer un couple constructeur / destructeur.

2.5.6 6. Toujours tester le retour de malloc / calloc

```
int* px = malloc(sizeof(int));

if (px == NULL) {
    fprintf(stderr, "Allocation failed\n");
    exit(EXIT_FAILURE);
}
```

Ne jamais supposer que l'allocation réussit.

2.5.7 7. malloc vs calloc

Pour des allocations multiples :

```
int* array = calloc(n, sizeof(int));
```

Avantages :

- Initialise la mémoire à 0
- Evite certaines erreurs d'initialisation

Toujours écrire :

```
int* array = malloc(n * sizeof(int));
```

et jamais :

```
int* array = malloc(n * 4);   // Mauvaise pratique
```

2.5.8 8. Initialiser la mémoire

Deux solutions :

- Utiliser `calloc`
- Utiliser `memset`: void *memset(void *adresseDeDepart, int valeurACopier, nbOctetARemplir n);

```
int* array = malloc(n * sizeof(int));
memset(array, 0, n * sizeof(int));
```

2.5.9 9. Après free : effacer l'adresse

```
free(px);
px = NULL;
```

Sinon, `px` devient un **dangling pointer** (pointeur pendu).

2.5.10 10. Passage par pointeur et const

Si un pointeur est utilisé uniquement en lecture :

```
void printValue(const int* px);
```

Cela garantit que la fonction ne modifiera pas la zone pointée.

Résumé des bonnes pratiques

- Toujours initialiser les pointeurs.
- Toujours tester les allocations.
- Toujours libérer ce qui a été alloué.
- Mettre le pointeur à `NULL` après `free`.
- Utiliser `sizeof(type)`.
- Fournir des fonctions de destruction.
- Utiliser `const` quand c'est possible.

La mémoire dynamique donne du pouvoir, mais exige de la discipline.

2.6 C01b — Pointers : Étude de cas : implémentation d'un tableau dynamique (vector)

Nous commençons maintenant une étude pratique visant à implémenter un tableau dynamique en C, c'est-à-dire l'équivalent :

- d'un `ArrayList` en Java,
- d'un `std::vector` en C++.

Le langage C ne fournit pas ce type de structure dans sa bibliothèque standard. Nous allons donc l'implémenter nous-mêmes, étape par étape.

1. Objectif

Nous voulons créer un tableau :

- dont la taille n'est pas connue à l'avance,
- et qui peut grandir dynamiquement.

Cela implique :

- une allocation dynamique,
- une réallocation lorsque la capacité est dépassée,
- une libération explicite.

2. Interface conceptuelle

On souhaite pouvoir écrire quelque chose comme :

```
vector v;

if (vector_create(&v) == 0) {
    vector_push(&v, 10);
    vector_push(&v, 42);
    vector_destroy(&v);
}
```

Remarques importantes :

- Les fonctions reçoivent un pointeur sur `vector` car elles modifient la structure.
- Si on alloue, on doit fournir une fonction de destruction.

3. Définition du type des éléments

Pour rendre l'implémentation flexible, on définit :

```
typedef int element_type;
```

En changeant une seule ligne, on pourrait transformer notre vector d'entiers en vector de `double`, par exemple.

4. Structure interne du vector

Nous définissons maintenant la structure :

```
typedef struct {
    size_t size;        // nombre d'elements effectivement stockes
    size_t capacity;    // nombre maximal d'elements alloues
    element_type* data; // pointeur vers la zone memoire
} vector;
```

Il est crucial de comprendre la différence entre :

- `size` : nombre réel d'éléments utilisés,
- `capacity` : nombre maximal d'éléments pouvant être stockés sans nouvelle allocation.

5. Sémantique mémoire

Supposons :

$$\texttt{capacity} = 100$$

Si un `int` occupe 4 octets, alors la zone allouée correspond à :

$$100 \times \texttt{sizeof(int)} = 400 \text{ octets}$$

Visuellement :

$$\underbrace{[\square][\square]\ldots[\square]}_{\text{capacity cases}}$$

Mais si seulement 5 éléments sont utilisés :

$$\texttt{size} = 5$$

Les autres cases sont simplement disponibles pour de futurs ajouts.

6. Stratégie d'extension

Pourquoi stocker une capacité plus grande que la taille réelle ?

Parce que si on réalloue à chaque insertion :

```
push -> realloc -> push -> realloc -> push -> realloc
```

cela serait très coûteux.

On adopte donc une stratégie par blocs :

- Allouer par exemple 100 éléments.
- Quand la capacité est atteinte, doubler la capacité.

$$100 \to 200 \to 400 \to 800$$

Cela permet d'avoir une complexité amortie efficace.

7. Problème : que faire au 101e élément ?

Supposons :

- `capacity = 100`
- `size = 100`

Lorsqu'on veut insérer le 101e élément, il faut agrandir la zone mémoire.

C'est ici qu'intervient :

$$\boxed{\texttt{realloc}}$$

8. Rôle de realloc

`realloc` permet de :

- agrandir ou réduire une zone mémoire existante,
- éventuellement déplacer la zone en mémoire,
- retourner une nouvelle adresse si nécessaire.

Principe général :

```
pointer = realloc(pointer, nouvelle_taille);
```

Attention :

- Si `realloc` échoue, il retourne `NULL`.
- L'ancien pointeur reste valide.
- Il faut donc toujours tester le retour.

9. Résumé conceptuel

Un vector dynamique contient :

- un nombre d'éléments utilisés,
- une capacité allouée,
- un pointeur vers la mémoire.

Lorsqu'on dépasse la capacité :

- on appelle `realloc`,
- on met à jour `capacity`,
- on continue l'insertion.

Idée clé :
La structure elle-même est statique, mais la mémoire pointée est dynamique et peut être déplacée.

Le vector contrôle dynamiquement sa propre zone mémoire.

2.7 C01b — Pointers : La fonction `realloc`

Pour redimensionner une zone mémoire déjà allouée dynamiquement, le langage C fournit la fonction :

`realloc`

Attention : ce n'est pas un `recalloc`. C'est un **re-malloc**. La nouvelle zone ajoutée n'est pas initialisée.

1. Prototype

```
void* realloc(void* ptr, size_t new_size);
```

Arguments :

- `ptr` : ancien pointeur (zone déjà allouée)
- `new_size` : nouvelle taille en octets

Valeur de retour :

- Nouvelle adresse (identique ou différente)
- `NULL` si échec

2. Comportement général

Cas 1 : agrandissement Si on augmente la taille :

```
int* tmp = realloc(ptr, 200 * sizeof(int));
```

Alors :

- Si de l'espace est disponible après la zone actuelle, l'adresse peut rester identique.
- Sinon, une nouvelle zone est allouée ailleurs.
- Les anciennes données sont copiées.
- L'ancienne zone est libérée automatiquement.

Important : la partie nouvellement allouée n'est pas initialisée.

3. Cas 2 : réduction

On peut aussi réduire la taille :

```
ptr = realloc(ptr, 50 * sizeof(int));
```

Dans ce cas :

- La zone peut être conservée telle quelle.
- Ou éventuellement déplacée.
- Les données au-delà de la nouvelle taille sont perdues.

4. Cas 3 : échec

Si `realloc` échoue :

- Il retourne `NULL`.
- L'ancienne zone mémoire reste inchangée.

```
ptr = realloc(ptr, new_size);   // DANGEREUX
```

Si `realloc` retourne `NULL`, vous perdez l'adresse originale → fuite mémoire.

```
int* tmp = realloc(ptr, new_size);

if (tmp == NULL) {
    // echec : ptr est toujours valide
    fprintf(stderr, "Reallocation failed\n");
} else {
    ptr = tmp;
}
```

Toujours utiliser un pointeur temporaire.

5. Cas particuliers

Bonne pratique

```
ptr = realloc(NULL, size);
```

Equivalent à :

```
ptr = malloc(size);
```

Même si c'est autorisé, il est préférable d'utiliser explicitement `malloc`.

```
realloc(ptr, 0);
```

Equivalent à :

```
free(ptr);
```

Là aussi, il est recommandé d'utiliser directement `free`.

6. Résumé du fonctionnement interne

realloc(ptr, 0)

- Peut agrandir ou réduire une zone mémoire.
- Peut déplacer la zone si nécessaire.
- Copie automatiquement les données existantes.
- Libère l'ancienne zone si déplacement.
- Ne touche pas à l'ancienne zone en cas d'échec.
- N'initialise pas la nouvelle mémoire ajoutée.

Point essentiel :

$$\boxed{\texttt{realloc} = \text{redimensionnement intelligent} + \text{copie automatique}}$$

Mais :

$$\boxed{\text{La mémoire ajoutée n'est PAS initialisée}}$$

Donc si vous agrandissez un tableau, vous devez initialiser manuellement la nouvelle partie.

2.8 C01b — Pointers : Étude de cas : implémentation complète du vector (constructeur, push, enlarge)

Nous détaillons maintenant les fonctions principales :

- le constructeur,
- le destructeur,
- la fonction `vector_push`,
- la fonction interne `vector_enlarge`.

L'objectif est de comprendre :

- le passage par référence du vector lui-même (cas 1),
- la gestion dynamique de la zone pointée (cas 3),
- l'écriture dite « atomique » pour éviter les corruptions en cas d'échec.

2.8.1 1. Constantes et types

```
#include <stdlib.h>
#include <stdio.h>
#include <limits.h>

typedef int element_type;

#define VECTOR_PADDING 128

typedef struct {
    size_t size;        // nombre d'elements utilises
    size_t allocated;   // capacite allouee
    element_type* content; // zone memoire dynamique
} vector;
```

2.8.2 2. Constructeur

Convention :

- retourne `NULL` en cas d'échec,
- sinon retourne le pointeur reçu (pour chaînage possible).

```
vector* vector_create(vector* v) {
    if (v == NULL) {
        return NULL;
    }

    vector result;
    result.size = 0;
    result.allocated = VECTOR_PADDING;
    result.content = calloc(VECTOR_PADDING, sizeof(element_type));

    if (result.content == NULL) {
        return NULL;
    }

    *v = result;  // ecriture atomique finale
    return v;
}
```

Pourquoi travailler sur `result` ?

Pour ne modifier `v` que lorsque tout s'est bien passé.

2.8.3 3. Destructeur

```
void vector_destroy(vector* v) {
    if (v == NULL) {
        return;
    }

    free(v->content);
    v->content = NULL;
    v->size = 0;
    v->allocated = 0;
}
```

2.8.4 4. Fonction interne : vector_enlarge

Objectif : augmenter la capacité de `VECTOR_PADDING` éléments.

Convention :

- retourne `NULL` en cas d'échec,
- sinon retourne `v`.

Protection supplémentaire : éviter l'overflow lors de la multiplication

$$allocated \times sizeof(element_type)$$

```
vector* vector_enlarge(vector* v) {
    if (v == NULL) {
        return NULL;
    }

    vector result = *v;

    // nouvelle capacite
    result.allocated += VECTOR_PADDING;

    // protection contre overflow
    if (result.allocated > SIZE_MAX / sizeof(element_type)) {
        return NULL;
    }

    element_type* tmp = realloc(
        result.content,
        result.allocated * sizeof(element_type)
    );

    if (tmp == NULL) {
        return NULL;
    }

    result.content = tmp;

    *v = result;   // ecriture atomique
    return v;
}
```

Pourquoi tester :

```
result.allocated > SIZE_MAX / sizeof(element_type)
```

Parce que tester :

$$allocated \times sizeof(T) > SIZE_MAX$$

pourrait déjà provoquer un overflow. On transforme donc mathématiquement :

$$x \times y > z$$

en

$$x > \frac{z}{y}$$

pour éviter le débordement.

2.8.5 5. Fonction vector_push

Convention choisie :

- retourne le nouveau nombre d'éléments,
- retourne 0 en cas d'échec.

```
size_t vector_push(vector* v, element_type value) {
    if (v == NULL) {
        return 0;
    }

    // Tant que la capacite est insuffisante
    while (v->size >= v->allocated) {
        if (vector_enlarge(v) == NULL) {
            return 0;
        }
    }

    // insertion a la position size
    v->content[v->size] = value;

    v->size++;

    return v->size;
}
```

2.8.6 6. Pourquoi un while et pas un if ?

On aurait pu écrire :

```
if (v->size == v->allocated)
```

Mais utiliser :

```
while (v->size >= v->allocated)
```

rend le code plus général :

- Permet d'ajouter plusieurs éléments d'un coup.
- Ne dépend pas d'une hypothèse « on ajoute toujours 1 élément ».
- Rend la fonction plus robuste.

Règle pratique :

Si le corps modifie la condition, demandez-vous si un while n'est pas préférable à un if

2.8.7 7. Logique complète du push

Deux cas :

Cas 1 : place disponible

- `size < allocated`
- on écrit directement à l'indice `size`

Cas 2 : plus de place

- appel à `vector_enlarge`
- éventuellement plusieurs fois
- puis insertion

Les indices valides sont :

$$0 \ldots size - 1$$

Le nouvel élément est inséré à :

$$index = size$$

Puis :

$$size \leftarrow size + 1$$

Pointeurs - 3

3.1 Chaînes de caractères en C : principes fondamentaux

3.1.1 1. Différence fondamentale avec Java

En C :

$$\boxed{\text{Il n'existe pas de type string.}}$$

Une « chaîne de caractères » est simplement :

$$\boxed{\text{un tableau de } \textit{char} \text{ terminé par } '\backslash 0'}$$

Il n'y a aucun objet spécial.

3.1.2 2. Convention de terminaison

La longueur n'est pas stockée explicitement. Elle est indiquée par une convention :

$$\boxed{\text{le caractère nul } '\backslash 0'}$$

Exemple en mémoire :

$$H \quad E \quad L \quad L \quad O \quad '\backslash 0'$$

C'est uniquement la présence du caractère nul qui marque la fin.

3.1.3 3. Littéraux de chaînes

```
"Bonjour"
```

est équivalent en mémoire à :

```
{'B','o','n','j','o','u','r','\0'}
```

Remarque :

- Les guillemets simples ‘’A’‘ → un seul caractère.
- Les guillemets doubles ‘”ABC”‘ → tableau de caractères + ‘”‘.

Si on veut écrire un guillemet dans la chaîne :

```
"Il a dit \"Bonjour\""
```

3.1.4 4. Initialisation d'un tableau

```
char s[] = "Bonjour";
```

Ici :

- s est un tableau.
- Il y a copie automatique des caractères.
- La taille inclut le caractère nul.

C'est équivalent à :

```
char s[] = {'B','o','n','j','o','u','r','\0'};
```

3.1.5 5. Pointeur n'est PAS string

```
char* s;
```

Ce n'est PAS une string.
C'est simplement :

$$\boxed{\text{un pointeur vers char}}$$

À ce stade, il pointe vers n'importe quoi. Il doit être :

- soit initialisé à `NULL`,
- soit alloué dynamiquement.

3.1.6 6. Allocation dynamique correcte

Toujours allouer :

$$n + 1$$

pour inclure le caractère nul.

```
size_t n = 10;
char* s = calloc(n + 1, sizeof(char));
```

Pourquoi `n + 1` ?
Parce que nous voulons manipuler n caractères utiles, et garder une place pour '\0'.

3.1.7 7. Erreur classique

```
char* s = "Bonjour";   // Mauvaise pratique
```

Pourquoi ?
Parce que :

- "Bonjour" est un littéral constant.
- Il est stocké en mémoire en lecture seule.
- s pointe vers une zone que vous n'avez pas le droit de modifier.

Si vous faites :

```
s[0] = 'A';   // comportement indefini
```

Vous risquez un crash.

3.1.8 8. Cas correct 1 : chaîne constante

Si vous ne modifiez jamais la chaîne :

```
const char* s = "Bonjour";
```

Ici :

- s pointe vers une zone constante.
- Toute tentative de modification est interdite par le compilateur.

3.1.9 9. Cas correct 2 : chaîne modifiable

Si vous voulez modifier la chaîne :

```
char* s = calloc(256, sizeof(char));
strcpy(s, "Bonjour");
```

Ici :

- La zone est allouée dynamiquement.
- On copie les caractères (y compris '\0').
- On peut modifier le contenu.

Exemple :

```
s[0] = 'A';
```

La chaîne devient :

$$"Aonjour"$$

3.1.10 10. Différence clé : copie vs pointeur

```
char s[] = "Bonjour";
```

→ Copie réelle des caractères.

```
char* s = "Bonjour";
```

→ Copie d'adresse uniquement.

Il n'y a aucune copie du contenu.

3.1.11 11. Règle essentielle

Initialisation de pointeur

En C, une chaîne est juste un tableau de char terminé par '\0'

Il n'existe aucun objet string.

Tout repose sur :

- la gestion explicite de la mémoire,
- le caractère nul,
- la distinction tableau vs pointeur.

3.2 Fonctions utilitaires pour les chaînes de caractères

Heureusement, la bibliothèque standard fournit de nombreuses fonctions pour manipuler les chaînes de caractères.

Toutes se trouvent dans :

```
#include <string.h>
```

Nous ne détaillons pas toutes les fonctions ici. Il est essentiel de consulter les man pages :

`man 3 string`

Nous allons surtout insister sur les pièges et les bonnes pratiques.

3.2.1 Toujours utiliser les versions avec `n`

Beaucoup de fonctions existent en deux versions :

- `strcpy` / `strncpy`
- `strcat` / `strncat`
- `strcmp` / `strncmp`

Recommandation : toujours utiliser les versions avec n

Pourquoi ?

Parce qu'elles permettent de contrôler explicitement la taille maximale manipulée.

3.2.2 Copie : `strncpy`

Prototype :

```
char* strncpy(char* dest, const char* src, size_t n);
```

- `dest` : destination (modifiable)
- `src` : source (const)
- `n` : nombre maximal de caractères copiés

Attention importante :

- Si la longueur de `src` n,
- alors `strncpy` ne garantit PAS l'ajout du caractère '\0'.

C'est un piège classique.

D'où la recommandation :

Toujours allouer $taille + 1$

et travailler avec une variable `taille` qui représente uniquement les caractères utiles.

3.2.3 Concaténation : `strncat`

```
char* strncat(char* dest, const char* src, size_t n);
```

Elle concatène au plus n caractères de `src` à la fin de `dest`.
Contrairement à `strncpy`, `strncat` garantit la terminaison par '\0'.
Toujours vérifier la taille disponible avant concaténation.

3.2.4 Comparaison : `strncmp`

```
int strncmp(const char* s1, const char* s2, size_t n);
```

Retour :

- < 0 si s1 $<$ s2 (ordre lexicographique)
- $= 0$ si égalité
- > 0 si s1 $>$ s2

Erreur classique :

```
if (s1 == s2)   // FAUX
```

Cela compare les adresses, pas le contenu.
La bonne façon :

```
if (strncmp(s1, s2, n) == 0) {
    // egalite
}
```

Souvent on écrit :

```
if (!strncmp(s1, s2, n)) {
    // egalite
}
```

3.2.5 Longueur : `strlen`

```
size_t strlen(const char* s);
```

Important :

- Une chaîne ne connaît pas sa longueur.
- `strlen` parcourt les caractères jusqu'à '\0'.

Complexité :

$$\boxed{O(n)}$$

Si vous appelez `strlen` dans une boucle, vous pouvez transformer un algorithme linéaire en quadratique.
Toujours stocker la longueur si elle est utilisée fréquemment.

3.2.6 Recherche de caractères

```
char* strchr(const char* s, int c);
```

Retourne un pointeur vers la première occurrence de `c` dans `s`, ou `NULL`.

`strrchr` Même chose, mais recherche de droite à gauche.

3.2.7 Recherche de sous-chaîne : `strstr`

```
char* strstr(const char* s1, const char* s2);
```

Retourne un pointeur vers la première occurrence de la sous-chaîne `s2` dans `s1`.

3.2.8 Entrées / sorties

Affichage

```
printf("%s", s);
```

puts

```
puts(s);
```

Ajoute automatiquement un retour à la ligne.

fputs

```
fputs(s, stdout);
```

N'ajoute PAS de retour à la ligne.

3.2.9 Lecture

scanf

```
scanf("%s", s);
```

Attention :

- Ne pas mettre `&s`
- `s` est déjà un pointeur

Mais `scanf` ne protège pas contre les débordements.

fgets (recommandé)

```
fgets(s, taille, stdin);
```

- Protège contre les dépassements
- Lit au maximum `taille - 1` caractères
- Ajoute toujours '\0'

3.3 Résumé des bonnes pratiques

- Toujours allouer `n + 1`.
- Toujours utiliser les versions avec `n`.
- Ne jamais comparer deux chaînes avec `==`.
- Toujours vérifier les tailles.
- Éviter `scanf` non contrôlé.
- Stocker la longueur si utilisée souvent.

En C, une chaîne est fragile : elle repose uniquement sur '\0'.

3.4 Pointeurs sur fonctions et généricité

En C, on peut pointer vers n'importe quelle zone mémoire, y compris vers du code.
Cela signifie :

On peut avoir des pointeurs sur fonctions.

3.4.1 Déclaration d'un pointeur sur fonction

Supposons la fonction :

```
double f(int x);
```

Pour déclarer un pointeur vers cette fonction :

```
double (*g)(int);
```

Règle :

On remplace le nom de la fonction par $(*nom_pointeur)$

3.4.2 Affectation

```
g = &f;    // syntaxe pedagogique
```

On peut aussi écrire :

```
g = f;     // equivalent
```

En C, le nom d'une fonction est déjà un pointeur vers la fonction.

3.4.3 Appel via pointeur

Version explicite :

```
double y = (*g)(10);
```

Version simplifiée :

```
double y = g(10);
```

Les deux sont équivalentes.

3.5 Passage de fonctions en argument

Exemple classique : intégration numérique.

```
typedef double (*fonction)(double);

double integrer(fonction f, double a, double b);
```

Appel :

```
double resultat = integrer(sin, 0.0, M_PI);
```

On passe ici un pointeur sur la fonction `sin`.

3.6 Généricité et pointeurs génériques

Le type :

```
void*
```

ne signifie pas « pointeur vers rien ».
Il signifie :

pointeur vers quelque chose dont on ignore le type

C'est un pointeur générique.

3.7 Exemple : tri générique avec qsort

Prototype standard :

```
void qsort(void* base,
           size_t nmemb,
           size_t size,
           int (*compar)(const void*, const void*));
```

Arguments :

- `base` : pointeur vers le tableau
- `nmemb` : nombre d'éléments
- `size` : taille d'un élément
- `compar` : fonction de comparaison

3.7.1 Pourquoi void* ?

Parce que le tableau peut contenir :

- des int,
- des double,
- des structures,
- etc.

On ne connaît pas le type à l'avance.

3.7.2 Exemple : tri d'un tableau d'entiers

```
int tab[] = {4, 1, 9, 2};
size_t n = 4;

qsort(tab, n, sizeof(int), compare_int);
```

3.7.3 Fonction de comparaison générique

Prototype imposé :

```
int compare_int(const void* a, const void* b)
```

Implémentation propre :

```
int compare_int(const void* a, const void* b) {
    const int* i = a;
    const int* j = b;

    if (*i < *j) return -1;
    if (*i > *j) return 1;
    return 0;
}
```

3.7.4 Pourquoi faire cette conversion ?

Parce que :

- `a` et `b` sont des `void*`
- On doit les interpréter comme des `int*`
- On ne modifie pas les valeurs → `const`

3.8 Comparaison pour des doubles

```
int compare_double(const void* a, const void* b) {
    const double* x = a;
    const double* y = b;

    if (*x < *y) return -1;
    if (*x > *y) return 1;
    return 0;
}
```

Même schéma, type différent.

3.9 Principe fondamental de la généricité en C

En C, la généricité repose sur :

- `void*` pour les données,
- pointeurs sur fonctions pour les comportements.

Contrairement aux templates C++, tout est fait manuellement.

$$\boxed{\text{void* + pointeur sur fonction = généricité en C}}$$

3.10 Points importants

- Toujours utiliser `const void*` dans les comparateurs.
- Toujours convertir vers le bon type avant déréférencement.
- Ne jamais modifier les données dans une fonction de comparaison.
- Toujours passer `sizeof(type)` à `qsort`.

3.11 Lecture des types complexes en C

À ce stade, nous avons vu :

- des pointeurs sur int,
- des pointeurs sur pointeurs,
- des fonctions retournant des pointeurs,
- des pointeurs sur fonctions,
- des tableaux de pointeurs,
- des combinaisons de tout cela.

Il devient alors essentiel de savoir lire des déclarations complexes.

3.11.1 Règle fondamentale de lecture

$$\boxed{\text{Toujours commencer par le nom.}}$$

Ensuite :

- Respecter les parenthèses.
- Lire d'abord ce qui est immédiatement autour du nom.
- Puis élargir progressivement vers l'extérieur.

Les parenthèses sont cruciales : elles définissent les priorités.

3.12 Exemple de type complexe

Considérons la déclaration suivante :

```
int *(*(*f())[])();
```

Comment lire cela ?

3.12.1 Étape 1 : partir du nom

Le nom est :

$$f$$

On regarde immédiatement ce qui suit :

```
f()
```

Donc :

$$f \text{ est une fonction}$$

3.12.2 Étape 2 : que retourne f ?

On regarde à gauche de `f()` :

```
(*f())
```

Donc :

$$f \text{ retourne un pointeur}$$

3.12.3 Étape 3 : vers quoi pointe ce pointeur ?

On regarde :

```
(*(*f())[])
```

Donc :

$$f \text{ retourne un pointeur vers un tableau}$$

3.12.4 Étape 4 : contenu du tableau

On regarde :

```
(*(*f())[])()
```

Cela signifie :

$$\text{tableau de pointeurs sur fonctions}$$

3.12.5 Étape 5 : que retournent ces fonctions ?

On regarde le type final :

```
int *
```

Donc :

les fonctions retournent un pointeur sur int

3.13 Lecture complète

On peut lire la déclaration ainsi :

f est une fonction qui retourne un pointeur vers un tableau de pointeurs sur fonctions retournant des pointeurs sur int

3.14 Conseil pratique essentiel

Même si vous êtes capable de lire ce genre de déclaration :

N'écrivez jamais cela directement.

Utilisez des `typedef`.

3.15 Réécriture avec typedef

```
typedef int* int_ptr;

typedef int_ptr (*func_ptr)(void);

typedef func_ptr func_array[];

typedef func_array* return_type;

return_type f(void);
```

Cette version est :

- plus lisible,
- plus maintenable,
- moins sujette aux erreurs.

3.16 Casting en C : valeurs et pointeurs

En C, le typage est souple. On peut convertir des expressions vers un autre type via un **casting**.
Syntaxe générale :

```
(type) expression
```

Il est essentiel de distinguer :

- le casting de valeurs,
- le casting de pointeurs.

3.17 Casting de valeurs

Exemple :

```
double x = 5.4;
int y = (int)x;
```

Ici :

- x est une valeur double.
- (int)x convertit la valeur.
- La partie fractionnaire est supprimée.

Donc :

$$y = 5$$

Le comportement est un arrondi vers zéro :

- 5.9 → 5
- -5.9 → -5

Ici, il y a réellement transformation de valeur.

3.18 Casting de pointeurs

C'est ici que les choses deviennent délicates.
Considérons :

```
double x = 5.4;
int* i = (int*)&x;
```

Important :

$$\boxed{\text{Le casting de pointeur ne modifie pas la mémoire.}}$$

Il change uniquement **l'interprétation** des bits.

3.18.1 Ce qui se passe réellement

En mémoire :

- x occupe sizeof(double) octets.
- Ces octets représentent 5.4 en format IEEE.
- i pointe vers la même adresse.
- Mais i lit seulement sizeof(int) octets.

Donc :

```
printf("%d\n", *i);
```

affichera une valeur incohérente correspondant à l'interprétation binaire des premiers octets du double.

Il n'y a aucune conversion de valeur.

3.19 Différence fondamentale

- Casting de valeur → conversion réelle.
- Casting de pointeur → relecture brute des bits.

Un casting de pointeur change la lecture, pas les données.

3.20 Casting et code générique

Le casting est utilisé légitimement dans :

- les fonctions génériques,
- les manipulations de void*,
- qsort.

Exemple :

```
int compare_int(const void* a, const void* b) {
    const int* i = a;
    const int* j = b;

    if (*i < *j) return -1;
    if (*i > *j) return 1;
    return 0;
}
```

Ici, le cast est nécessaire pour interpréter correctement la mémoire.

3.21 Casting de pointeurs sur fonctions

On peut aussi caster des pointeurs sur fonctions.

Exemple avec `qsort` :

```
qsort(tab, n, sizeof(Person),
      (int (*)(const void*, const void*))compare_person);
```

Cependant, cette écriture est difficile à lire.

Une alternative plus propre :

```
int compare_person(const void* a, const void* b) {
    const Person* p1 = a;
    const Person* p2 = b;
```

```
    ...
}
```

Puis :

```
qsort(tab, n, sizeof(Person), compare_person);
```

Plus lisible et plus sûr.

3.22 Casting et void*

En C :

- On peut convertir automatiquement vers void*.
- On peut convertir automatiquement depuis void*.

Exemple :

```
int* p1;
void* p2;

p2 = p1;   // pas de cast necessaire
p1 = p2;   // pas de cast necessaire en C
```

3.22.1 Erreur courante avec malloc

Beaucoup écrivent :

```
int* p = (int*)malloc(sizeof(int));
```

Ce cast est inutile en C.
Écriture correcte :

```
int* p = malloc(sizeof(int));
```

Pourquoi ?

- malloc retourne un void*.
- La conversion vers int* est implicite en C.

Ajouter un cast :

- surcharge le code,
- peut masquer des erreurs d'inclusion de headers.

Pointers - 4

4.1 Pointeurs et tableaux

Nous allons clarifier le lien fondamental entre pointeurs et tableaux.

On peut écrire :

```
double* ptr = calloc(3, sizeof(double));
```

Ici :

- ptr est un pointeur vers double.
- On alloue dynamiquement de la place pour 3 doubles.
- La mémoire est initialisée à 0.

Important :

$$\boxed{*ptr \text{ lit uniquement le premier élément}}$$

Même si la zone allouée contient 3 doubles, `*ptr` accède seulement au premier.

4.2 Accès via la syntaxe tableau

On peut accéder aux éléments ainsi :

```
ptr[0]
ptr[1]
ptr[2]
```

Rappel fondamental :

$$\boxed{*ptr \equiv ptr[0]}$$

Plus généralement :

$$ptr[i] \equiv *(ptr + i)$$

La notation tableau est donc une simple syntaxe pour l'arithmétique des pointeurs.

4.3 Différence fondamentale : pointeur vs tableau

Considérons maintenant :

```
double tab[3];
```

Différence essentielle :

- Un pointeur est une variable contenant une adresse.
- Un tableau est une zone mémoire continue.
- Le nom du tableau n'est pas une variable contenant une adresse.

Un tableau :

- est alloué statiquement (souvent sur la stack),

- ne possède pas de case mémoire supplémentaire pour stocker une adresse,
- ne peut pas être réaffecté.

On peut voir un tableau comme :

$$\boxed{T[N] \approx T * const}$$

C'est un pointeur constant vers une zone mémoire continue.

4.4 Mémoire : stack vs heap

Rappel organisation mémoire :

- Stack : variables locales, tableaux statiques.
- Heap : mémoire dynamique via malloc/calloc.

Exemple :

```
double P1[N][M];
double* P2[N];
double** P3;
```

4.5 Cas 1 : tableau bidimensionnel

```
double P1[N][M];
```

Caractéristiques :

- Alloué entièrement sur la stack.
- Mémoire continue.
- Contient N × M doubles consécutifs.

Accès :

```
P1[i][j]
```

Mémoire contiguë.

4.6 Cas 2 : tableau de pointeurs

```
double* P2[N];
```

Ici :

- P2 est un tableau de N pointeurs.
- Chaque P2[i] peut pointer vers une zone différente.
- Les lignes peuvent avoir des tailles différentes.

Allocation typique :

```
for (size_t i = 0; i < N; ++i) {
    P2[i] = calloc(M, sizeof(double));
}
```

La mémoire des lignes est sur le heap. Le tableau des pointeurs est sur la stack.

4.7 Cas 3 : double pointeur

```
double** P3 = calloc(N, sizeof(double*));

for (size_t i = 0; i < N; ++i) {
    P3[i] = calloc(M, sizeof(double));
}
```

Ici :

- P3 est un pointeur vers pointeur.
- La structure complète est sur le heap.
- Les blocs ne sont pas nécessairement contigus.

4.8 Différences cruciales

4.8.1 Continuité mémoire

- P1 : mémoire entièrement continue.
- P2 : N pointeurs contigus, lignes non contiguës.
- P3 : tout dynamique, non garanti contigu.

4.8.2 Allocation

- P1 : allocation statique.
- P2 : mix stack + heap.
- P3 : entièrement heap.

4.8.3 Dimensions variables

- P1 : toutes les lignes ont M colonnes.
- P2 et P3 : chaque ligne peut avoir une taille différente.

4.9 Attention importante

$$\boxed{T[N][M] \neq T**}$$

Un tableau bidimensionnel n'est pas un double pointeur.

Un tableau multidimensionnel :

- est alloué de façon continue,
- ressemble à un pointeur constant vers bloc continu.

Il ne faut jamais confondre :

```
int a[N][M];
```

avec :

```
int** a;
```

Ce sont des objets fondamentalement différents.

4.10 Arithmétique des adresses

Exemple :

```
printf("%ld\n", &P1[1][2] - &P1[0][0]);
```

Dans le cas de P1 :

$$\text{distance} = 1 \times M + 2$$

Car la mémoire est continue.

Dans le cas de P2 ou P3 :

- La distance peut être très grande.
- Les blocs peuvent être éloignés en mémoire.
- Stack et heap peuvent être très séparés.

4.11 Résumé final

- Un tableau 1D ressemble à un pointeur constant.
- Un tableau 2D n'est PAS un double pointeur.
- Les tableaux multidimensionnels sont continus.
- Les doubles pointeurs ne garantissent aucune continuité.
- Toujours distinguer allocation stack et heap.

$$\boxed{\text{Continuité mémoire} = \text{différence fondamentale}}$$

4.12 Arithmétique des pointeurs

L'arithmétique des pointeurs permet de se déplacer efficacement dans une zone mémoire.

On peut utiliser :

- +, -
- ++, –
- différence de pointeurs

4.13 Principe fondamental

Si `p` est un pointeur sur un objet de type T :

$$\boxed{p + 1 \text{ pointe vers l'objet suivant de type T}}$$

Il ne faut pas penser en octets.

Il faut penser :

$$\text{objet suivant}$$

En réalité, le compilateur ajoute :

$$sizeof(T)$$

à l'adresse, mais il ne faut pas raisonner ainsi.

4.14 Exemple : parcours d'un tableau

```
int tab[N];

int* p = tab;
int* const end = tab + N;

while (p < end) {
    printf("%d\n", *p);
    ++p;
}
```

Analyse :

- tab est le début du tableau.
- tab + N pointe juste après le dernier élément.
- ++p avance d'un entier.

Important :

$$\boxed{*p \equiv p[0]}$$

4.15 Pointeur constant

```
const int* const end = tab + N;
```

Cela signifie :

- On ne modifie pas les valeurs pointées.
- On ne modifie pas le pointeur lui-même.

4.16 Exemple classique : parcours de chaîne

Considérons :

```
char* s = "ABC";
char* p = s;
char c;
```

Boucle typique :

```
while ((c = *p++) != '\0') {
    putchar(c);
}
```

4.17 Comprendre *p++

Question :

Est-ce :

```
(*p)++
```

ou

```
*(p++)
```

Réponse :

$$\boxed{*p++ \equiv *(p++)}$$

Car l'opérateur postfixe ++ a une priorité plus élevée que *.

4.18 Ordre des opérations

```
c = *p++;
```

Équivalent à :

1. Lire la valeur pointée par p.
2. Affecter à c.
3. Incrémenter p.

Donc :

lecture puis déplacement

4.19 Différence avec (*p)++

```
(*p)++;
```

Ici :

- On modifie la valeur pointée.
- On n'avance pas le pointeur.

On incrémente dans l'espace des valeurs, pas dans l'espace des pointeurs.

4.20 Différence entre *p++ et *++p

```
*p++
```

Lecture puis déplacement.

```
*++p
```

Déplacement puis lecture.

Ordre inverse.

4.21 Pourquoi utiliser une variable temporaire ?

Dans :

```
while ((c = *p++) != '\0') {
    ...
}
```

Si on écrivait simplement :

```
while (*p++ != '\0') {
    putchar(*p);   // ERREUR logique
}
```

On lirait un caractère en avance.
La variable temporaire garantit que l'on travaille sur le bon caractère.

4.22 Différence de pointeurs

On peut soustraire deux pointeurs :

```
ptrdiff_t d = p2 - p1;
```

Le résultat :

$$\boxed{\text{nombre d'objets entre p1 et p2}}$$

Type retourné :

$$\boxed{ptrdiff_t}$$

Ce n'est pas un int.
Pourquoi ?

- Peut être négatif.
- Doit pouvoir représenter de grandes distances mémoire.

4.23 Indexation et équivalences

```
tab[i]
```

est strictement équivalent à :

```
*(tab + i)
```

Et même :

```
i[tab]
```

Car :

$$a[b] \equiv *(a + b)$$

Même si cette écriture est déconseillée.

4.24 Tableaux et passage aux fonctions

Quand on écrit :

```
void f(int tab[]);
```

Le compilateur traduit en :

```
void f(int* tab);
```

La taille du tableau n'est pas connue.

Un tableau passé en argument devient un pointeur.

4.25 Résumé

- p + 1 signifie objet suivant.
- *p++ = lecture puis déplacement.
- (*p)++ = modification de la valeur.
- p2 - p1 retourne un ptrdiff_t.
- tab[i] = *(tab + i).
- Les tableaux passés aux fonctions deviennent des pointeurs.

Toujours raisonner en objets, jamais en octets.

4.26 L'opérateur `sizeof`

L'opérateur `sizeof` renvoie la taille en mémoire (en octets) du type ou de l'expression donnée.

Syntaxe :

```
sizeof(type)
sizeof(expression)
```

4.27 Point fondamental

L'expression passée à sizeof n'est PAS évaluée.

Exemple piégeux :

```
int i = 10;
sizeof(i++);   // i n'est PAS incrementé
```

Ceci est strictement équivalent à :

```
sizeof(i);
```

4.28 Exemples simples

```
sizeof(double)   // taille d'un double
sizeof(int)      // taille d'un int
```

Si :

```
int tab[N];
```

Alors :

```
sizeof(tab)
```

renvoie :

$$N \times sizeof(int)$$

4.29 Calcul du nombre d'éléments d'un tableau

Idiom classique :

```
size_t n = sizeof(tab) / sizeof(tab[0]);
```

Cela fonctionne car :

$$\frac{N \times sizeof(int)}{sizeof(int)} = N$$

4.30 Attention : tableau vs pointeur

Considérons :

```
int tab[1000];
int* t = tab;
```

Alors :

```
sizeof(tab)   // 1000 * sizeof(int)
sizeof(t)     // sizeof(int*)
```

$$\boxed{sizeof(t) \neq sizeof(tab)}$$

4.31 Erreur classique

Dans une fonction :

```
void f(int t[]) {
    size_t n = sizeof(t) / sizeof(t[0]);
}
```

Ceci est faux.

Pourquoi ?

Parce que :

```
int t[]
```

est transformé par le compilateur en :

```
int* t
```

Donc :

$$sizeof(t) = sizeof(int*)$$

Le résultat sera 1, 2 ou autre petite valeur, mais certainement pas la taille réelle du tableau.

4.32 Règle absolue

$$\boxed{Untableauneconnatjamaissataille.}$$

Toujours passer la taille en argument :

```
void f(int t[], size_t n);
```

4.33 Bug classique avec l'arithmétique

Erreur fréquente :

```
int tab[1000];
int* end = tab + sizeof(tab);
```

Problème :

$$sizeof(tab) = 1000 \times sizeof(int)$$

Donc on avance beaucoup trop loin.
Correct :

```
int* end = tab + sizeof(tab)/sizeof(tab[0]);
```

4.34 Autre bug typique

```
for (int* p = tab; p < tab + sizeof(tab); ++p) {
    ...
}
```

Ici, on dépasse largement la mémoire du tableau.
Cela peut provoquer :

- lecture hors limites,
- écriture hors limites,
- buffer overflow.

4.35 Bonne pratique

Toujours écrire :

```
size_t n = sizeof(tab)/sizeof(tab[0]);

for (int* p = tab; p < tab + n; ++p) {
    ...
}
```

4.36 Résumé

- sizeof renvoie une taille en octets.
- L'expression n'est jamais évaluée.
- sizeof(tab) fonctionne uniquement si tab est un vrai tableau.
- Dans une fonction, un tableau devient un pointeur.
- Toujours passer explicitement la taille.
- Attention aux dépassements lors de tab + sizeof(tab).

sizeof est simple, mais source de nombreux bugs subtils.

4.37 Bonus : Flexible Array Member

Le **Flexible Array Member** (FAM) est un pattern avancé, souvent utilisé en programmation système.

L'idée est de créer une structure contenant :

- une taille,
- un tableau dynamique stocké directement à la suite en mémoire.

4.38 Motivation

Classiquement, on écrirait :

```
typedef struct {
    size_t size;
    double* data;
} vector;
```

Dans ce cas :

- La structure contient un pointeur.
- Les données sont ailleurs dans le heap.
- Il y a une indirection supplémentaire.

Le Flexible Array Member permet d'éviter cela.

4.39 Définition

```
typedef struct {
    size_t size;
    double data[];
} vector;
```

Important :

$$\boxed{Letableauflexibledoittreledernierchampdelastructure.}$$

Il n'a pas de taille déclarée.

4.40 Principe mémoire

La structure contient :

- la partie fixe (size),
- puis une zone contiguë pour les données.

On alloue alors :

$$sizeof(vector) + (n - 1) \times sizeof(double)$$

Pourquoi $n - 1$?
Parce que `sizeof(vector)` inclut déjà la place pour un élément de `data` (selon l'ancienne convention avec data[1]). Avec la forme moderne data[], il faut simplement allouer l'espace total nécessaire.

4.41 Allocation sécurisée

On doit éviter les overflows lors du calcul.

```
vector* create_vector(size_t n) {
    if (n == 0) return NULL;

    if (n > (SIZE_MAX - sizeof(vector)) / sizeof(double)) {
        return NULL;
    }

    vector* v = malloc(sizeof(vector) +
                       n * sizeof(double));

    if (v == NULL) return NULL;

    v->size = n;

    return v;
}
```

4.42 Utilisation

Une fois alloué :

```
vector* v = create_vector(10);

v->data[0] = 1.0;
v->data[1] = 2.0;
```

Les données sont stockées immédiatement après la structure en mémoire, de manière contiguë.

4.43 Avantage principal

Comparons les deux approches.

4.43.1 Avec pointeur

```
typedef struct {
    size_t size;
    double* data;
} vector;
```

Mémoire :

- structure sur heap,
- tableau séparé sur heap,
- une indirection supplémentaire.

4.43.2 Avec flexible array member

```
typedef struct {
    size_t size;
    double data[];
} vector;
```

Mémoire :

- une seule allocation,
- données contiguës,
- pas d'indirection,
- meilleure localité cache.

4.44 Schéma conceptuel

Structure classique :

$$[size][ptr] \rightarrow [data...]$$

Flexible array :

$$[size][data...data...data...]$$

4.45 Limites

- Pattern plus avancé.
- Doit être alloué dynamiquement.
- Le champ flexible doit être en dernier.
- Impossible d'avoir d'autres champs après.

4.46 Pourquoi c'est utilisé

- Code système.
- Structures variables.
- Optimisation mémoire.
- Protocoles réseau.

4.47 Résumé

- Permet d'avoir une structure + données contiguës.
- Évite une indirection.
- Améliore la localité mémoire.
- Nécessite une allocation calculée avec soin.

Flexible Array Member = structure extensible en mémoire contiguë.